```
#include <RBDdimmer.h>
#include "DHT.h"
#include <math.h>

// --- Pins ---
#define outputPin 12
#define zerocross 14
#define DHTPIN    4
#define DHTTYPE   DHT22

// --- Taille des tableaux ---
#define N 4000

// --- Objets ---
dimmerLamp dimmer(outputPin, zerocross);
DHT dht(DHTPIN, DHTTYPE);

// --- Paramètres du modèle ---
float w0 = 0.0499, w1 = 0.6055, w2 = 0.5768;
float w3 = 0.005,  w4 = 0.0084;
float v = 0;

// --- Paramètres de commande (minimum de variance) ---
float alpha  = 1.0;
float lambda = 0.1;
float mu     = 0.1;
float x, dJ;
int   imax   = 30;

// --- Variables globales ---
float Power;
int   k = 3;

float u[N];
float y[N];
float y_p[N];
float yc[N];

// ------------------------------------------------
void setup() {
  Serial.begin(115200);
  dht.begin();

  dimmer.begin(NORMAL_MODE, ON);
  dimmer.setPower(0);

  // Initialisation des tableaux
  for (int i = 0; i < N; i++) {
    u[i]   = 0.0;
    y[i]   = 0.0;
    y_p[i] = 0.0;

  }
  for (int i = 0; i < N; i++) {
    if (i < N / 2) {
        yc[i] = 0.35;
    } else {
        yc[i] = 0.45;
    }
}
}

// ------------------------------------------------
void loop() {

  // --- Lecture de la température ---
  float t = dht.readTemperature();
  if (isnan(t)) {
    Serial.println("Erreur DHT");
    delay(2000);
    return;
  }

```

```
    // --- Normalisation ---
    y[k] = t / 100.0;

    // --- Modèle d'identification (prédiction courante) ---
    v      = -w0 + w1*y[k-1] + w2*y[k-2] + w3*u[k-1] + w4*u[k-2];
    y_p[k] = tanh(v);

    // --- Initialisation de la variable d'optimisation ---
    x = u[k - 1];

    // --- Boucle d'optimisation (descente de gradient) ---
    for (int i = 0; i < imax; i++) {
      v          = -w0 + w1*y[k] + w2*y[k-1] + w3*x + w4*u[k-1];
      y_p[k + 1]  = tanh(v);
      // Correction de prédiction par erreur de modèle
      y_p[k + 1] += alpha * (y[k] - y_p[k]);
      // Gradient de la fonction de coût
      float df    = 1.0f - pow(tanh(v), 2);
      dJ = -2.0f * (yc[k+1] - y_p[k+1]) * df * w3
           + 2.0f * lambda * (x - u[k-1]);
      // Mise à jour par gradient
      x -= mu * dJ;
    }

    // --- Mise à jour de la commande ---
    u[k] = x;

    // --- Saturation [0, 1] ---
    if (u[k] > 1.0f) u[k] = 1.0f;
    if (u[k] < 0.0f) u[k] = 0.0f;

    // --- Conversion en pourcentage ---
    Power = u[k] * 100.0f;
    if (Power > 100.0f) Power = 100.0f;
    if (Power <   0.0f) Power =   0.0f;

    // --- Envoi au dimmer ---
    dimmer.setPower((int)Power);

    // --- Affichage série ---
    Serial.print(k);
    Serial.print(" ");
    Serial.print(t, 2);
    Serial.print(" ");
    Serial.println(Power, 2);

    delay(2000);

    // --- Incrément avec protection débordement ---
    k++;
    if (k >= N - 1) k = 3;   // réserve 2 indices d'avance (k+1 utilisé)
  }
```